

- This cache coherency protocol is based on a distributed collection of L1 caches that all can be both read from or written to.
 - For the purposes of this document, it seems easier to rely on having each L1 caches just be implemented the same as an L1 dcache for simplicity's sake.
- All of the L1 caches will be hooked up to a central directory.
- If multiple L1 caches share a cache line, then we have the following situations:
 - The directory will forward any write by a CPU to any shared cache line to each L1 cache that has a copy of said shared cache line.
 - A priority arbiter will block a CPU from writing to its attached L1 dcache while the central directory is forwarding a write to that L1 dcache.
 - Write forwarding is done in a serialized manner.
 - I was thinking of having a (new?) module that is similar to a reorder buffer, sort of, to implement the serialization.
 - This module would have, I suppose, one write port per L1 dcache (which is most likely the same number as the number of CPU cores). (L1 icaches don't have to worry about this specific thing, which should help with all three of performance in cycles, performance as in clock rate, and also area).
 - The directory needs to avoid actually sending evicted L1 dcache lines to the “higher level bus” (i.e. RAM, most likely) **iff** the L1 dcache line is located in another L1 dcache.
 - When an L1 cache fetches a fresh copy of a cache line upon a cache miss the directory will look to see whether that cache line is already in another L1 cache, and if so, the directory will forward that request to the first L1 cache it can find that has a copy of that cache line.
 - I think this means I need to add a third “request” interface to the priority arbiter in front of each L1 cache's “request” port. The priorities are as follows, with lower numbers in the list indicating a higher priority than higher numbers in the list:
 1. Writes to an L1's shared cache lines.
 2. Fetching a shared cache line (i.e. upon an L1 cache having a miss).
 3. The L1 cache's attached CPU's usual/typical requests.
 - Three ports like this seem like a good idea.
 - About atomics:
 - I'm going with load-reservation/store-conditional for my atomic primitives.
 - About lr instructions:
 - An lr instruction will reserve an address **in the directory**.
 - Any write to that address (even a non-sc write!!!!) (or to the cache line that shares that address I guess, maybe?) will clear the reservation's bit.
 - About sc instructions:
 - Upon success, a sc instruction will be forwarded via the normal write-forwarding mechanism that is used for plain store instructions.
 - LcvBus needs to be modified to have the following optional signals for when atomics are supported by a particular instance of the bus.
 - host-to-device (request) bus channels:
 - isAtomic: Bool
 - The isWrite field is used to determine the kind of atomic:
 - if(isWrite == False): lr (load reservation)
 - if(isWrite == True): sc (store conditional)
 - The byteSize field is used to determine the size of the atomic, but I might be able to require that byteSize **only** be for either the bus-word-width atomics, or for double-bus-word-width atomics.

- I think RISC-V also supports smaller sizes for atomics though?
- device-to-host (response) bus channels:
 - `atomicSuccess: Bool`
 - This indicates that the `sc` instruction was a success.
- This design all together means that my existing L1 dcache **does not** need to have its implementation changed, which may well save me a bunch of rework. However, this approach **does** seem to complicate the impl of the central directory module.